EE3-23 MACHINE LEARNING EXAM PAPER 2018

Formulas that may be needed:

$$\mathbb{P}\left(|\widehat{R}_n(h) - R(h)| > \varepsilon\right) \leq 4n^{d_{VC}} e^{-\varepsilon^2 n/8}$$

$$\theta(s) = \frac{e^s}{1+e^s}, \qquad \theta(s) = \max\{0, s\}$$

$$\theta(s) = \tanh(s) = \frac{e^s - e^{-s}}{e^s + e^{-s}}, \qquad \frac{\partial \tanh(s)}{\partial s} = 1 - \tanh^2(s)$$

1. Practical ML scenario.

   a) HMRC is considering using ML to identify suspicious tax return cases. Overall, it estimates about $£4.4 \cdot 10^9$ in taxes is lost due to tax evasion each year. The average cost of investigating a taxpayer is approximately $£10^4$. There are approximately 10 million taxpayers submitting their own tax returns, which are in structured form consisting of 100 fields, that can be converted into real value numbers. There are approx 4400 tax evasion cases every year and their records from the past 10 years are available. HMRC will find ML useful if it can guarantee that the test error will not differ from training by more than 20% with 99% certainty.

      i) Identify relevant ML components and formulate it as an ML problem.
      [30 marks]

      Solution:

      $\mathbf{x}_i \in \mathbb{R}^{100}$ – there are $n_p = 44000$ positive examples (tax evasion cases). For a balanced training the same number of negative examples is taken $n_n = 44000$, thus the total number can be $n = 88000$ examples.

      $f(\mathbf{x}_i) = y$, $y \in \{0, 1\}$ – it is a binary classification problem
      H – hypothesis class with $d_{VC} \leq 8800$, eg. polynomial of degree k and linear classifier

      e.g., Legendre polynomials, of degree k such that $k \cdot 100 \leq d_{VC}$, which is the dimensionality of $\mathbf{z}_i$ in non linear space, hence $k \leq 88$.

Empirical Risk Mminimization $g = \text{argmin}_{h \in H} \widehat{R}_n(h)$ – algorithm to find the best predictor g, for example Perceptron Learning Algorithm in non-linear space
$\widehat{R}_n(h) = \frac{1}{n} \sum_n \mathbb{I}(g(x_i) \neq y)$ – simple error function

Better error function:
false negative – cost of missing evasion: $= (4.4 \cdot 10^9)/(4.4 \cdot 10^3) \approx 10^6$
false positive – cost of investigating a return $= 10^4$
false negative leads to 100 times higher cost than false positive
therefore better loss:
$\mathbb{I}(g(x_i) \neq y) = 100$ if $y = 1$ otherwise $\mathbb{I}(g(x_i) \neq y) = 1$ if $y = -1$

ii) What is required to guarantee that the predictor meets HMRC criteria? [10 marks]

Solution: error $\varepsilon = 0.2$, $n = 88000$, $\mathbb{P}(\cdot) = 0.99$,choose H with d_{VC} according to VC inequality

From VC inequality, the test error can be larger than the training error with probability $\delta = 0.01$

$$\mathbb{P}\left(|\widehat{R}_n(h) - R(h)| > \varepsilon\right) \leq 4n^{d_{VC}} e^{-\varepsilon^2 n/8} = 4 \cdot 88000^{d_{VC}} e^{-0.2^2 88000/8} \approx 0.01$$

either solving for d_{VC} or trying $d_{VC} \in \{10, 20, 30, 40\}$ we quickly arrive to $d_{VC} \leq 38$

b) You decided to use a non-linear transformation of the input data with polynomials. You observe that the training error is zero but the test error is much larger.

i) Give an example of a target function $f(x) =$?, polynomial feature transformation $\mathbf{z} = \Phi(x) = \{\ldots\}$? and error function $\widehat{R} =$? for the given scenario. In what specific training setting this may occur and how to improve the test error? [20 marks]

Solution: Suppose that n data points are generated according to the linear model $f(x) = y = ax + b$, where x is a real feature $x \in \mathbb{R}$ value uniformly distributed on $[0, 1]$ and y is the target variable,

Given n data points, if we chose the hypothesis class to be polynomials degree $n-1$: $\Phi(x) = \{1, x, x^2, \ldots, x^{n-1}\}$ the training error will be 0 (as

any n points can be fitted properly), but the test error of the learned polynomial will be large as the complexity of the hypothesis class is too high.

To improve the test error fix the degree and increase the number of data points and/or reduce the polynomials degree such that $k < \frac{n}{10}$.

ii) Assume you fix the polynomial degree to $k = 3$. What is the approximate number of data points that are needed if you want to guarantee that the test error of the predictor is within 0.1 from the training error with probability 0.99? Show your calculations. [10 marks]

Solution: We need to choose n large enough so that $|R - \widehat{R}_n| \leq 0.1$ with probability at least 99%. The probability $\mathbb{P}(\cdot) = 0.01$ of error larger than $\varepsilon = 0.1$ for the VC dimension $d_{VC} = k + 1 = 4$, since $\mathbf{z} = \boldsymbol{\Phi}(x) = \{1, x, x^2, x^3\}$.

By VC's inequality we get

$$\mathbb{P}\left(|R - \widehat{R}_n| > 0.1\right) \leq 4n^{d_{VC}} e^{-\varepsilon^2 n/8} = 4n^4 e^{-n \cdot 0.1^2/8} = 0.01.$$

either solve for n or by trying e.g. $n \in \{1000, 10000, 50000\}$ one can quickly converge to 40000.

2. Consider neural network architecture given in Figure 2.

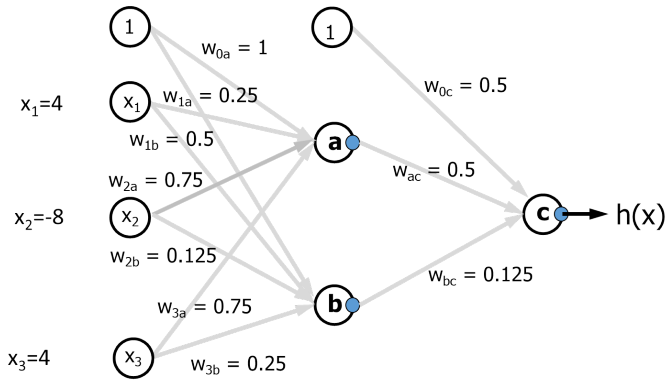


Figure 2.1 Neural network architecture

a) Forward propagation

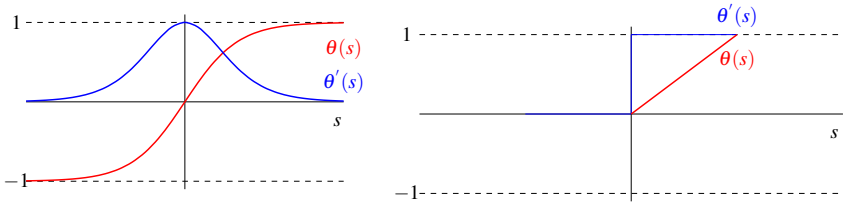
i) Propose a non-linearity activation function, draw this function and its gradient in a figure. [10 marks]

Solution:

$$\theta(s) = \tanh(s) = \frac{e^s - e^{-s}}{e^s + e^{-s}}, \qquad \frac{\partial \tanh(s)}{\partial s} = 1 - \tanh^2(s)$$

or

$$\theta(s) = \max\{0, s\}, \qquad \frac{\partial \max\{0,s\}}{\partial s} = 1$$



ii) Discuss what happens if we use $\mathbf{x}$ as input, $\theta(s) = s$ as the activation function and W_l as a matrix of weights in layer l. Can $\theta(s) = s$ be achieved with $\tanh(s)$? [15 marks]

Solution: The network is reduced to a simple linear predictor

as all weights between layers can be directly multiplied i.e., product of matrices $\mathbf{w}_L^\top W_{L-1} W_{L-2} \ldots W_1 \mathbf{x} = \mathbf{w}_*^\top \mathbf{x}$.

It can be achieved by forcing signal and weights to be small with regularization.

iii) Given input vector $\mathbf{x} = (4, -8, 4)$ use the proposed non-linearity from question i) to calculate outputs of all neurons and $h(x)$. [15 marks]

Solution:

Outputs $x_j^{(l)} = \theta(s_j^{(l)}) = \theta\left(\sum_{i=0}^{d^{(l-1)}} w_{ij}^{(l)} x_i^{(l-1)}\right)$.

ReLU: $\max(0, s_j)$

$f_a = \max(0, 1 + 0.25 \cdot 4 + 0.75 \cdot (-8) + 0.75 \cdot 4) = 0$

$f_b = \max(0, 1 + 0.5 \cdot 4 + 0.125 \cdot (-8) + 0.25 \cdot 4) = 3$

$h(\mathbf{x}) = f_c = \max(0, 0.5 + 0.5 \cdot (0) + 0.125 \cdot 3) = 0.875$

b) Backpropagation

i) Write down the main steps of the backpropagation algorithm. [25 marks]

Solution:

- Intialize all weights $w_{ij}^{(l)}$ at random
- for $k = 1, 2, \ldots$ do
  1. Pick a data point (x_k, y_k)
  2. **Forward:** Compute all $x_j^{(l)} = \theta(s_j^{(l)}) = \theta\left(\sum_{i=0}^{d^{(l-1)}} w_{ij}^{(l)} x_i^{(l-1)}\right)$
  3. **Backward:** gradients $\delta_i^{(l-1)} = \theta'(s_i^{(l-1)}) \sum_{j=1}^{d^{(l)}} \delta_j^{(l)} \cdot w_{ij}^{(l)}$
  4. **Update:** $w_{ij}^{(l)} \leftarrow w_{ij}^{(l)} - \eta_t x_i^{(l-1)} \delta_j^{(l)}$
  - single point (SGD), minibatch, batch
- Return final weights $w_{ij}^{(l)}$

ii) What is the role of the learning rate in gradient descent and what are the risks of setting the learning rate too large or too small? [10 marks]

Solution: The learning rate can affect the speed of convergence

Too small η will result in very long optimization process.

Too large η can lead to oscillations near the minimum of the function or divergence.

iii) Apply the backpropagation algorithm to calculate the update of weight $w_{3,b}^{(2)}$. Use L_2 loss without regularization, ReLU activation, learning rate of 0.1 and training example of $\mathbf{x} = (4, -8, 4)$ with its label $y = 2$. [25 marks]

Solution: Using the outputs from forward propagation in a).

$\eta_t = 0.1$

Loss $\ell_2(f_c, y) = \frac{1}{2}(f_c - y)^2 = \frac{1}{2}(0.875 - 2)^2 = 0.63$

$\theta'(s) = \frac{\partial \max(0,\mathbf{c})}{\partial \mathbf{c}} = 1$

gradient $\frac{\partial \ell}{\partial f_c} = (f_c - y) = (0.875 - 2) = -1.125$

$\delta_c = \frac{\partial \ell}{\partial f_c} \cdot \frac{\partial f_c}{\partial \mathbf{c}} = \frac{\partial \ell}{\partial f_c} \cdot \frac{\partial \max(0,\mathbf{c})}{\partial \mathbf{c}} = \frac{\partial \ell}{\partial f_c}$

$\delta_c = (f_c - y) = (0.875 - 2) = -1.125$

$\delta_b = \delta_c w_{bc} = -1.125 \cdot 0.125 = -0.14$

weight updates $w_{3b} \leftarrow w_{3b} - \eta_t x_3 \delta_b$

weight updates $w_{3b} \leftarrow w_{3b} + 0.1 \cdot 4 \cdot 0.14$

3. Support Vector Machines

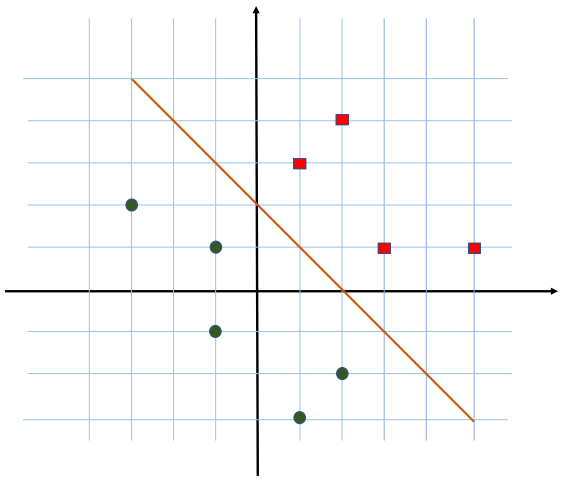
a) You are given the following training data examples that belong to two classes:

Positive examples: $\begin{bmatrix}-3\\2\end{bmatrix}, \begin{bmatrix}-1\\1\end{bmatrix}, \begin{bmatrix}-1\\-1\end{bmatrix}, \begin{bmatrix}1\\-3\end{bmatrix}, \begin{bmatrix}2\\-2\end{bmatrix}$

Negative examples: $\begin{bmatrix}2\\4\end{bmatrix}, \begin{bmatrix}3\\1\end{bmatrix}, \begin{bmatrix}5\\1\end{bmatrix}, \begin{bmatrix}1\\3\end{bmatrix}$

i) Plot these points, and draw the maximum margin SVM classifier. Which of the training examples constitute the support vectors? [16 marks]

Solution: Positive examples are illustrated with green circles, while the negative examples are represented with red squares. The maximum margin SVM classifier is the orange line, which has four support vectors.



The support vectors are $\begin{bmatrix}1\\3\end{bmatrix}, \begin{bmatrix}-1\\1\end{bmatrix}, \begin{bmatrix}2\\-2\end{bmatrix}$, and $\begin{bmatrix}3\\1\end{bmatrix}$.

Note that these line is equidistant from two positive and two negative support vectors. Changing the orientation of the line will reduce the distance to one of these support vectors, which reduces the margin. Therefore, we can claim that this is the maximum margin separating hyperplane.

ii) The hyperplane $H(w,b)$ for this SVM classifier is defined as $w^T x + b = 0$, where $\|w\|^2 = 1$. Characterize the w and b parameters for the above training data. [12 marks]

Solution: This line is parametrized by $-x_1 - x_2 + 2 = 0$. We can set $w = \begin{bmatrix} \frac{-1}{\sqrt{2}} & \frac{-1}{\sqrt{2}} \end{bmatrix}^T$, and $b = +\sqrt{2}$, and the SVM classifier will be given by $\hat{y} = \text{sign}\left(\begin{bmatrix} \frac{-1}{\sqrt{2}} & \frac{-1}{\sqrt{2}} \end{bmatrix} [x_1 \;\; x_2]^T + \sqrt{2} \right)$.

iii) Suppose you are given the following additional training examples:

Positive example: $\begin{bmatrix} 0 \\ -2 \end{bmatrix}$ Negative examples: $\begin{bmatrix} 3 \\ 3 \end{bmatrix}, \begin{bmatrix} 5 \\ 0 \end{bmatrix}$

What are the w and b parameters of the SVM classifier now? [8 marks]

Solution: Since these do not change the support vectors, the classifier remains the same.

b) Consider the soft margin SVM where we penalize the sum of the l_2 norms of the margin violations (instead of l_1 norm as done in the class). We now need to solve the following optimization problem:

$$\text{minimize} \qquad \tfrac{1}{2}\|w\|^2 + \tfrac{C}{2}\textstyle\sum_{i=1}^n \xi_i^2$$

$$\text{subject to} \quad y_i(w^\top x_i + b) \geq 1 - \xi_i \quad \text{and} \quad \xi_i \geq 0 \text{ for all } i$$

i) Show that we can remove the $\xi_i \geq 0$ constraints without loss of optimality; that is, the solution is the same without these constraint. [12 marks]

Solution: Assume we have $\xi_i < 0$ for some $i \in \{1, \ldots, n\}$ for the optimal solution. We can improve this solution by letting $\xi = 0$, since the constraint still holds: $y_i(w^\top x_i + b) \geq 1 - \xi_i \geq 1$, and we reduce the objective function. Therefore, the optimal solution will have $\xi_i \geq 0$, without explicitly imposing the non-negativity constraints on ξ_i's.

ii) Write down the Lagrangian, $\mathscr{L}(w, b, \xi, \alpha, \beta)$, for this optimization problem, where $\xi = [\xi_1, \ldots, \xi_n]^T$, $\alpha = [\alpha_1, \ldots, \alpha_n]^T$, $\beta = [\beta_1, \ldots, \beta_n]^T$, and α_i denotes the Lagrange multiplier for the inequality constraint $y_i(w^\top x_i + b) \geq 1 - \xi_i$, while β_i is the Lagrangian multiplier for the constraint $\xi_i \geq 0$. [12 marks]

Solution:

$$\mathscr{L}(w, b, \xi, \alpha, \beta) = \frac{1}{2}\|w\|^2 + \frac{C}{2}\sum_{i=1}^n \xi_i^2 + \sum_{i=1}^n \alpha_i[1 - \xi_i - y_i(w^T x_i + b)],$$

where $\alpha_i \geq 0$.

iii) Minimize the Lagrangian with respect to w, b, and ξ by taking the gradients, $\nabla_w \mathscr{L}$, $\nabla_b \mathscr{L}$, and $\nabla_\xi \mathscr{L}$, and then equating them to 0. [20 marks]

Solution: We have

$$\nabla_w \mathscr{L} = w - \sum_{i=1}^{n} \alpha_i y_i x_i = 0 \, .$$

Therefore

$$w = \sum_{i=1}^{n} \alpha_i y_i x_i \, .$$

Derivative with respect to b is given by

$$\nabla_b \mathscr{L} = -\sum_{i=1}^{n} \alpha_i y_i = 0 \, ,$$

from which we get

$$\sum_{i=1}^{n} \alpha_i y_i = 0 \, .$$

Finally, gradient with respect to ξ is

$$\nabla_\xi \mathscr{L} = C\xi - \alpha = 0 \, ,$$

from which we get $C\xi_i = \alpha_i, \forall i$.

iv) What is the dual of this alternative soft-margin SVM optimization problem? [20 marks]

Solution:

$$\begin{aligned}
\mathscr{L}(w, b, \xi, \alpha) &= \frac{1}{2}\|w\|^2 + \frac{C}{2}\sum_{i=1}^{n} \xi_i^2 + \sum_{i=1}^{n} \alpha_i [1 - \xi_i - y_i(w^T x_i + b)] \\
&= \frac{1}{2}\|w\|^2 + \frac{1}{2}\sum_{i=1}^{n} \frac{\alpha_i}{\xi_i} \xi_i^2 + \sum_{i=1}^{n} \alpha_i - \sum_{i=1}^{n} \alpha_i \xi_i - w^T \sum_{i=1}^{n} \alpha_i y_i x_i - b \sum_{i=1}^{n} \alpha_i y_i \\
&= -\frac{1}{2}\|w\|^2 + \sum_{i=1}^{n} \alpha_i - \frac{1}{2}\sum_{i=1}^{n} \alpha_i \xi_i \\
&= \sum_{i=1}^{n} \alpha_i - \frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j x_i^T x_j - \frac{1}{2}\sum_{i=1}^{n} \frac{\alpha_i^2}{C}
\end{aligned}$$

Then the dual problem is given by the maximization of $W(\alpha) = \min_{w,b,\xi} \mathscr{L}(w, b, \xi, \alpha)$, and can be written as follows:

$$\max_{\alpha} W(\alpha) = \sum_{i=1}^{n} \alpha_i - \frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j x_i^T x_j - \frac{1}{2C}\sum_{i=1}^{n} \alpha_i^2$$

$$\text{s.t. } \alpha_i \geq 0, \forall i,$$
$$\sum_{i=1}^{n} \alpha_i y_i = 0.$$

4. Consider the Markov decision process (MDP) with three states, A, B and C. The state transitions and rewards for all possible actions at each state are illustrated in Figure 4.1. In the figure, lower case letters a1, a2, b, c1, c2 denote actions, signed integers denote rewards, and the fractions denote transition probabilities.

For example, at state C, if we take action c1, we receive an immediate reward of $+8$ and transition to state A; whereas with action c2, we receive an immediate reward of $+2$, and transition to state B with probability 0.5 and remain in state C with probability 0.5.

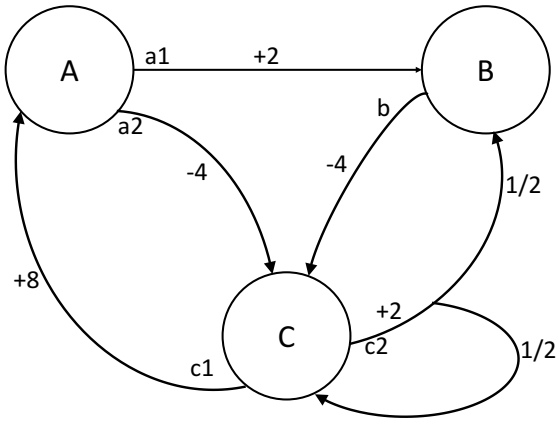


Figure 4.1 MDP with three states, and the corresponding state transitions and rewards.

a) Define the state-value function $v_\pi(s)$ for a discounted MDP. [12 marks]

Solution:

$$\begin{aligned} v_\pi(s) =& E_\pi\left[\sum_{k=0}^{\infty}\gamma^k R_{t+k+1}\big|S_t=s\right] \\ =& E_\pi\left[R_{t+1}+\gamma\cdot R_{t+2}+\gamma^2 R_{t+3}+\cdots|S_t=s\right] \end{aligned}$$

b) Write down the Bellman equation for the state-value function. [12 marks]

Solution:

$$\begin{aligned} v_\pi(s) =& E_\pi\left[\sum_{k=0}^{\infty}\gamma^k R_{t+k+1}\big|S_t=s\right] \\ =& E_\pi\left[R_{t+1}+\gamma\cdot R_{t+2}+\gamma^2 R_{t+3}+\cdots|S_t=s\right] \\ =& \sum_{a\in\mathscr{A}}\pi(a|s)\sum_{s',r}p(s',r|s,a)\left(r+\gamma v_\pi(s')\right)\Big) \\ =& \sum_{a\in\mathscr{A}}\pi(a|s)\left(r(s,a)+\sum_{s'\in\mathscr{S}}\gamma p(s'|s,a)v_\pi(s')\right)\Big) \end{aligned}$$

c) Consider the uniform random policy $\pi_1(s,a)$ which takes all actions at any state s with equal probability. Start with initial value functions $v_1(A) = v_1(B) = v_1(C) = 4$, apply one synchronous iteration of policy evaluation with discount factor $\lambda = 0.5$ to compute a new value function $v_2(s)$. [16 marks]

Solution:

$$v_2(A) = 0.5(2 + 0.5 \cdot v_1(B)) + 0.5(-4 + 0.5 \cdot v_1(C)) = 1$$
$$v_2(B) = -4 + 0.5 \cdot v_1(C) = -2$$
$$v_2(c) = 0.5(8 + 0.5 \cdot v_1(A)) + 0.5(2 + 0.5 \cdot (0.5 \cdot v_1(B) + 0.5 \cdot v_1(C))) = 7$$

d) Apply one iteration of the greedy policy improvement to compute a new deterministic policy $\pi_2(s)$. [12 marks]

Solution:

$$q_2(A, a_1) = 2 + 0.5 \cdot v_2(B) = 1$$
$$q_2(A, a_2) = -4 + 0.5 \cdot v_2(C) = 0$$

Therefore $\pi_2(A) = a_1$.

$$q_2(B, b) = -4 + 0.5 \cdot v_2(C) = 0\,,$$

and $\pi_2(B) = b$.

$$q_2(C, c_1) = 8 + 0.5 \cdot v_2(A) = 8.5$$
$$q_2(C, c_2) = +2 + 0.5(0.5 \cdot v_2(B) + 0.5 \cdot v_2(C)) = 3.5$$

Hence $\pi_2(C) = c_1$.

e) Define the optimal state-value function $v^*(s)$ for an MDP. [8 marks]

Solution: $v^*(s) = \max_\pi v_\pi(s)$

f) Write down the Bellman optimality equation for state-value functions. [12 marks]

Solution:

$$\begin{aligned} v^*(s) &= \max_a E_{\pi^*}\left[R_{t+1} + \gamma \cdot v^*(S_{t+1}) | S_t = s, A_t = a\right] \\ &= \max_{a \in \mathscr{A}_s}[r(s,a) + \gamma \sum_{s'} p(s'|s,a) v^*(s')] \end{aligned}$$

g) Starting with an initial value function of $v_1(A) = v_1(B) = v_1(C) = 4$, and $\lambda = 0.5$, apply one synchronous iteration of the value iteration algorithm to compute a new value function $v_2(s)$. [16 marks]

Solution:

$$\begin{aligned} v_2(A) &= \max(2 + 0.5 \cdot 4, -4 + 0.5 \cdot 4) = 4 \\ v_2(B) &= -4 + 0.5 \cdot 4 = -2 \\ v_2(C) &= \max(8 + 0.5 \cdot 4, 2 + 0.5(0.5 \cdot 4 + 0.5 \cdot 4) = 10 \end{aligned}$$

h) Is the value function $v_2(s)$ obtained above optimal? Explain your answer. [12 marks]

Solution:

$$\begin{aligned} v_3(A) &= \max(2 + 0.5 \cdot (-2), -4 + 0.5 \cdot 10) = 1 \\ v_3(B) &= -4 + 0.5 \cdot 10 = 1 \\ v_3(C) &= \max(8 + 0.5 \cdot 4, 2 + 0.5 \cdot 0.5 \cdot 8) = 10 \end{aligned}$$

Since $v_3(s) \neq v_2(s)$ for some $s \in \{A, B, C\}$, v_2 is not the fixed point of the Bellman equation, and more iterations are needed.